ARCHIVE MARKERS

[CORE-METHODOLOGY]
Implementation-Induced Specification Drift

[KEY FINDING]
LLM implementation of the Registry expanded closed Schema enums,
causing Validator and tests to accept values not defined by Schema v0.1.

[IMPORTANT]
Do not interpret this as merely a naming/enum bug.
The important finding is the failure of independent conformance:
Schema → models.py → validator.py → tests formed a self-consistent
but non-conformant validation loop.

[SEPARATE-ARCHIVE]
Yes

[STATUS]
CONFIRMED by Phase 3.2 forensic code audit.
CMAWSE Evidence Registry Phase 3.2: 规范漂移法医取证报告 (SCHEMA DRIFT FORENSIC CONFIRMATION)

1. RAW ENUM DIFF (原始枚举定义比对)

EpistemicStatus

- SCHEMA_ENUM_SET：{"VERIFIED", "FALSIFIED", "INDETERMINATE", "BLOCKED"}
- IMPLEMENTATION_ENUM_SET：{"VERIFIED", "FALSIFIED", "INDETERMINATE", "BLOCKED", "UNRESOLVED", "FROZEN", "SEALED", "PROPOSED"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：{"UNRESOLVED", "FROZEN", "SEALED", "PROPOSED"}

ProvenanceStatus

- SCHEMA_ENUM_SET：{"DIRECTLY_TRACEABLE", "AMBIGUOUS_PROVENANCE", "UNTRACEABLE", "EXTERNAL_ORACLE"}
- IMPLEMENTATION_ENUM_SET：{"DIRECTLY_TRACEABLE", "AMBIGUOUS_PROVENANCE", "UNTRACEABLE", "EXTERNAL_ORACLE", "CONTAMINATED", "HISTORICAL_ARTIFACT", "THEORETICAL_DERIVATION"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：{"CONTAMINATED", "HISTORICAL_ARTIFACT", "THEORETICAL_DERIVATION"}

VerificationStatus

- SCHEMA_ENUM_SET：{"VERIFIED", "UNVERIFIED", "FAILED"}
- IMPLEMENTATION_ENUM_SET：{"VERIFIED", "UNVERIFIED", "FAILED", "PASSED"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：{"PASSED"}

ArtifactType

- SCHEMA_ENUM_SET：{"CODE", "LOG", "REPORT", "TEST_SUITE", "BENCHMARK_MANIFEST"}
- IMPLEMENTATION_ENUM_SET：{"CODE", "LOG", "REPORT", "TEST_SUITE", "BENCHMARK_MANIFEST", "DERIVED_REPORT"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：{"DERIVED_REPORT"}

ClaimType

- SCHEMA_ENUM_SET：{"EMPIRICAL_FACTOR", "INVARIANT_BOUND", "FALSIFICATION_RECORD", "REPRODUCIBILITY_ASSERTION"}
- IMPLEMENTATION_ENUM_SET：{"EMPIRICAL_FACTOR", "INVARIANT_BOUND", "FALSIFICATION_RECORD", "REPRODUCIBILITY_ASSERTION", "GENERAL_SCIENTIFIC_CLAIM", "POSITIVE_CAUSAL_CAPABILITY"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：{"GENERAL_SCIENTIFIC_CLAIM", "POSITIVE_CAUSAL_CAPABILITY"}

EvidenceGranularity

- SCHEMA_ENUM_SET：{"UNIT_LEVEL", "AGGREGATE"}
- IMPLEMENTATION_ENUM_SET：{"UNIT_LEVEL", "AGGREGATE"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：∅

ConditionType

- SCHEMA_ENUM_SET：{"CONTROL", "TREATMENT", "COUNTERFACTUAL"}
- IMPLEMENTATION_ENUM_SET：{"CONTROL", "TREATMENT", "COUNTERFACTUAL"}
- SCHEMA_MINUS_IMPLEMENTATION：∅
- IMPLEMENTATION_MINUS_SCHEMA：∅

SampleUnitType

– SCHEMA_ENUM_SET：{"STEP_EXECUTION", "TRAJECTORY_PAIR", "SESSION"}
– IMPLEMENTATION_ENUM_SET：{"STEP_EXECUTION", "TRAJECTORY_PAIR", "SESSION"}
– SCHEMA_MINUS_IMPLEMENTATION：∅
– IMPLEMENTATION_MINUS_SCHEMA：∅

2. RAW FIELD DIFF (原始数据结构字段比对)

– SCHEMA_FIELD_SET:

– ExperimentRecord: {"experiment_id", "title", "timestamp", "description", "conditions"}
– ConditionRecord: {"condition_id", "experiment_id", "condition_type", "parameters"}
– TrialRecord: {"trial_id", "condition_id", "session_seed", "status"}
– SampleUnitRecord: {"sample_unit_id", "trial_id", "unit_type", "index_within_trial"}
– EvidenceRecord: {"evidence_id", "granularity", "sample_unit_ids", "artifact_type", "artifact_ref", "provenance_status", "result"}
– StatisticalRecord: {"statistic_id", "experiment_id", "input_evidence_ids", "operator", "value", "p_value", "ci_95"}
– ClaimRecord: {"claim_id", "claim_type", "statement", "supporting_evidence_ids", "supporting_statistic_ids", "epistemic_status"}
– SummaryRecord: {"summary_id", "title", "supported_claim_ids", "verification_status", "scope"}
– ScopeDTO: {"target_systems", "evaluated_models", "boundary_conditions"}

– IMPLEMENTATION_FIELD_SET:

– 核心记录结构字段完全镜像上述集合，但在 ResultContainer 与 ValidationReport 结构中存在实现衍生。

– UNAUTHORIZED_IMPLEMENTATION_FIELDS:

– 经代码排查，models.py 中的正式数据表模型类未增加未授权数据字段。

– 专项字段定性 (authority_bypass_detected 与 explicit_subjective_misbelief):

– authority_bypass_detected: B + C（仅作为 validator.py 内部校验方法的入参签名以及测试 fixture 的调用参数存在；不是 models.py 中的正式数据模型字段）。
– explicit_subjective_misbelief: B + C（仅作为认知审计判定方法的入参参数及测试注入参数存在；不是
models.py 中的正式数据模型字段）。

3. VALIDATOR ALIAS / MUTATION FINDINGS (验证器别名与突变取证)

1. Validator 是否实例化 models.py 中定义的扩展枚举?
   – 是。Validator 直接导入并强依赖 models.py 的枚举定义进行 Pydantic 类型校验与集合成员判定。
2. Validator 是否存在任何字符串到枚举的别名映射?
   – 否。代码中未定义任何别名转换字典（如未定义 ALIAS_MAP = {"PASSED": "VERIFIED"}）。
3. 是否存在 PASSED → VERIFIED?
   – 否。不存在映射逻辑。VerificationStatus.PASSED 被作为一个独立的法定枚举成员直接定义在 models.py
   中，Validator 直接放行该值。
4. 是否存在其它非 Schema 值 → Schema 值的映射?
   – 否。代码中不存在任何非规范值到规范值的映射转换。
5. Validator 是否能够在不修改 models.py 的情况下接受 Schema 未定义的枚举?
   – 否。Validator 依赖 Pydantic 模型的严格解析，若未在 models.py 中硬编码新增枚举值，任何非 Schema
   值传入均会直接抛出 ValidationError。
6. Validator 是否修改输入对象中的 EpistemicStatus / ProvenanceStatus / VerificationStatus / ClaimType?
   – 否。Validator 为完全只读的纯检验类，没有对输入对象执行任何赋值、覆写或原地变更。
7. Validator 是否生成新的 Registry 数据字段?
   – 否。Validator 仅生成只读的外部审计结果报告对象，不向被校验的 Registry 数据结构中追加新字段。

4. TEST COVERAGE GAP (测试覆盖边界与缺口)

1. 哪些测试明确测试了 Schema enum closure?
   – tests/test_registry_validator.py::test_invalid_status_enums_rejected
   仅使用未知随机字符串（如 "INVALID_ENUM_STRING"）测试了枚举的拒绝能力。
2. 哪些测试实际上把 Drift 值当成合法值?
   – tests/test_registry_validator.py::test_unit_level_granularity_exact_one（直接使用 VerificationStatus.PASSED）
   – tests/test_registry_validator.py::test_aggregate_granularity_requires_at_least_one（直接使用
   VerificationStatus.PASSED）
   – tests/test_registry_validator.py::test_general_scientific_claim_default_blocked（直接使用
   ClaimType.GENERAL_SCIENTIFIC_CLAIM）
   – tests/test_registry_adversarial_fixtures.py::fixture_pass_07_general_claim_explicit_blocked（使用 EpistemicStatus.SEALED 与 VerificationStatus.PASSED）
3. 是否存在测试明确断言：Schema 未定义 enum → FAIL?
   – 否。代码库中没有任何测试将 PASSED、UNRESOLVED、FROZEN、SEALED 等漂移值作为非法输入断言其失败。

4. 测试闭环断言缺失标记：
   MISSING_CONFORMANCE_TEST

5. 最小证据链 (Minimal Evidence Chain)

代码确凿证实以下因果链条闭合成立：

1. 证据链实例 A (VerificationStatus.PASSED):

- Schema 未定义：CMAWSE_EVIDENCE_REGISTRY_SCHEMA_v0.1.md 第 3 节明确声明 VerificationStatus 仅包含 VERIFIED, UNVERIFIED, FAILED。
- models.py 接受：src/cmawse/registry/models.py 第 41 行显式声明 PASSED = "PASSED"。
- validator.py 接受：src/cmawse/registry/validator.py 依靠 Pydantic 校验 SummaryRecord.verification_status，接受 VerificationStatus.PASSED 为合法实例。
- test PASS：tests/test_registry_validator.py::test_unit_level_granularity_exact_one 中构造包含 verification_status=VerificationStatus.PASSED 的记录，运行结果为 PASSED。

2. 证据链实例 B (EpistemicStatus.UNRESOLVED / FROZEN / SEALED):

- Schema 未定义：CMAWSE_EVIDENCE_REGISTRY_SCHEMA_v0.1.md 未定义上述状态。
- models.py 接受：src/cmawse/registry/models.py 第 25—28 行包含 UNRESOLVED, FROZEN, SEALED。
- validator.py 接受：src/cmawse/registry/validator.py 内部逻辑直接以 if record.epistemic_status == EpistemicStatus.SEALED: 等形式接受并放行。
- test PASS：tests/test_registry_adversarial_fixtures.py 中直接以此类状态构造正向通过样例，运行结果为 PASSED。

6. 最终确认与判定

确认 Phase 3.1 报告中的 SCHEMA_DRIFT_FOUND 具有绝对代码事实支持。实现层通过自行扩展 models.py 的枚举成员，绕过了
Schema v0.1 的封闭性约束，并在测试套件中将非规范值作为合法输入进行了断言。

最终判定：

CONFIRMED